

Maximum Flow

Max-Flow Min-Cut Theorem · Ford–Fulkerson
· Edmonds–Karp · Bipartite Matching

Algorithm Engineering

Lecture 24 · 120 minutes

Part I — Foundations (~25 min)

- Flow networks, flows, flow value
- Antiparallel edges, multiple sources/sinks
- Residual networks & augmenting paths
- Flow augmentation (Lemma 24.1)

Part II — The Big Theorem (~20 min)

- Cuts of a flow network
- Max-Flow Min-Cut Theorem

Part III — Algorithms (~40 min)

- Ford–Fulkerson method & analysis
- Bad case for naive augmenting-path choice
- Edmonds–Karp algorithm (BFS augmenting paths)
- $O(VE)$ augmentation bound, $O(VE^2)$ total

Part IV — Application (~10 min)

- Maximum bipartite matching via max flow
- Correctness & $O(VE)$ running time

Part I — Flow Networks

Networks, capacities, and the flow-conservation law

Motivation: Material Flowing Through a Network

Model a directed graph as a **flow network**: material flowing from a **source** s , where it is produced, to a **sink** t , where it is consumed.

Applications of the same abstraction:

- Liquids flowing through pipes (capacity = gallons/hour)
- Parts moving through assembly lines
- Electrical current through a circuit (capacity constraint $\leftrightarrow$ Kirchhoff's current law)
- Information through a communication network
- (Later) machines matched to tasks — a combinatorial problem in disguise

Key Idea

Maximum-flow problem: compute the greatest rate at which material can be shipped from s to t without violating any edge's capacity. Find bottleneck links.

Definition — Flow network

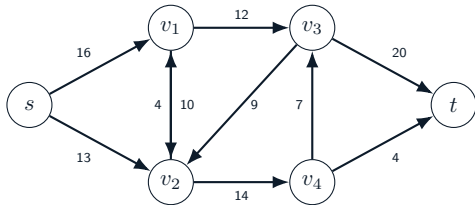
A **flow network** $G = (V, E)$ is a directed graph in which every edge $(u, v) \in E$ has a nonnegative capacity $c(u, v) \geq 0$.

- If $(u, v) \notin E$, we define $c(u, v) = 0$ for convenience.
- No self-loops are allowed.
- **No antiparallel edges:** if $(u, v) \in E$ then $(v, u) \notin E$.
- Two distinguished vertices: source s and sink t .
- Every vertex lies on *some* path $s \rightsquigarrow v \rightsquigarrow t$.

Immediate consequence: since every vertex other than s has at least one entering edge, $|E| \geq |V| - 1$.

These are modelling *restrictions*, not limitations of the theory — the next few slides show how to remove each restriction by transforming the network.

Example: The Lucky Puck Trucking Network



Source (s) produces material; Sink (t) collects them. Each edge (u, v) is a path with capacity $c(u, v)$ items/day.

The network cannot be changed. what is the **largest** number p of item/day that can be pushed from s to t ?

Definition — Flow

A **flow** in G is a real-valued function $f : V \times V \rightarrow \mathbb{R}$ satisfying:

(i) **Capacity constraint.** For all $u, v \in V$:

$$0 \leq f(u, v) \leq c(u, v).$$

(ii) **Flow conservation.** For all $u \in V - \{s, t\}$:

$$\sum_{v \in V} f(v, u) = \sum_{v \in V} f(u, v).$$

- $f(u, v)$ is called the flow *from* u *to* v .
- If $(u, v) \notin E$ then necessarily $f(u, v) = 0$.
- Flow conservation **flow in equals flow out** at every internal vertex.

Definition — Value of a flow

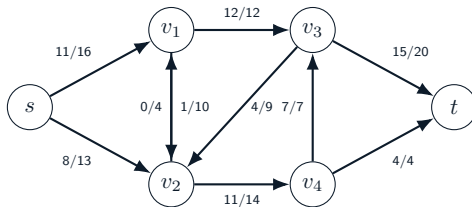
$$|f| = \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s),$$

i.e., total flow *out of* the source minus flow *into* the source. (Here $|\cdot|$ denotes flow value, *not* absolute value or cardinality.)

Typically a flow network has no edges into s , so the second sum is 0 — but we keep it in the definition because, once we introduce *residual networks*, flow into s can become positive.

The Value of a Flow II

The Maximum-Flow Problem: given G with source s and sink t , find a flow f of maximum value $|f|$.

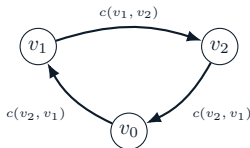


A flow f with $|f| = 19$. Slash notation $f(u, v)/c(u, v)$ — *not* division.

Removing Restriction 1: Antiparallel Edges

Edges like (v_1, v_2) and (v_2, v_1) would both appear as — antiparallel edges, which violate our definition.

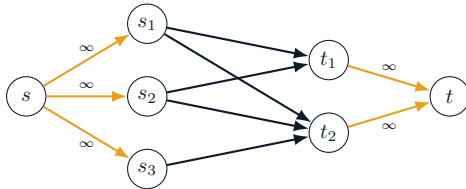
Fix: split one of the two edges by inserting a new vertex.



- Choose one edge, say (v_2, v_1) ; add new vertex v_0 .
- Replace (v_2, v_1) with (v_2, v_0) and (v_0, v_1) , each with capacity $c(v_2, v_1)$.
- Resulting network has *no* antiparallel edges.

Removing Restriction 2: Multiple Sources and Sinks

Real instances may have several sources $\{s_1, \dots, s_m\}$ (factories) and several sinks $\{t_1, \dots, t_n\}$ (warehouses). This is **no harder** than ordinary max flow.



- Add a **supersource** s , edges (s, s_i) with $c(s, s_i) = \infty$ for each i .
- Add a **supersink** t , edges (t_j, t) with $c(t_j, t) = \infty$ for each j .
- Any flow in the multi-source/sink network corresponds to a flow of *identical value* in this single-source/sink network, and vice versa

What we now have

- A precise model: flow network = capacities + conservation law.
- A precise goal: maximize $|f| = \sum_v f(s, v) - \sum_v f(v, s)$.
- Two modelling restrictions (no antiparallel edges, single source/sink) can always be removed by network transformations that preserve the optimal value.

Still missing: an actual *algorithm*. The key computational idea is to iteratively push more flow along paths from s to t — but naive greedy pushing can get stuck at a suboptimal flow. We need a way to *undo* bad earlier decisions. That mechanism is the **residual network**, coming up next.

Residual Networks & Augmenting Paths

Undoing flow decisions to make progress

Why We Need Residual Networks

Suppose we greedily push flow along some s - t path, saturating an edge. Later we may discover a better global assignment requires *less* flow on that edge.

Key idea

Allow an algorithm to **push flow backward** along an edge that already carries flow — this is equivalent to *decreasing* that edge's flow, which may free up capacity to route more flow elsewhere.

The **residual network** G_f records, for every ordered pair of vertices, how much *additional* flow could be pushed:

- forward along an existing under-saturated edge, or
- backward, cancelling flow already sent.

Residual Capacity: Formal Definition

Definition — Residual capacity

For a flow network $G = (V, E)$, flow f , and $u, v \in V$:

$$c_f(u, v) = \begin{cases} c(u, v) - f(u, v) & \text{if } (u, v) \in E, \\ f(v, u) & \text{if } (v, u) \in E, \\ 0 & \text{otherwise.} \end{cases}$$

Since G has no antiparallel edges, $(u, v) \in E \implies (v, u) \notin E$, so *exactly one* case applies to each ordered pair.

Worked example. If $c(u, v) = 16$ and $f(u, v) = 11$:

- $f(u, v)$ can increase by up to $c_f(u, v) = 16 - 11 = 5$ before violating the capacity constraint.
- Up to 11 units can be sent back from v to u : $c_f(v, u) = 11$.

Definition — Residual network

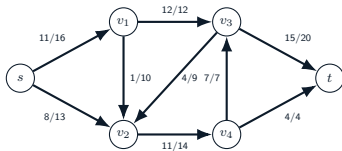
Given $G = (V, E)$ and flow f , the residual network is $G_f = (V, E_f)$ where

$$E_f = \{(u, v) \in V \times V : c_f(u, v) > 0\}.$$

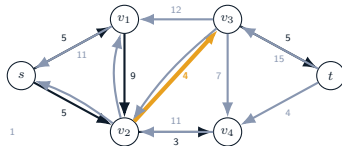
- Every edge of E_f is either an edge of E or the *reversal* of one, so $|E_f| \leq 2|E|$.
- G_f behaves like a flow network with capacities c_f , **except** it may contain antiparallel edges — that's fine, we never claimed G_f is itself a valid flow network by our original definition, only that we can speak of flows *within* it.
- A “flow in G_f ” means a function obeying capacity constraint and conservation *w.r.t.* c_f .

Example: Building the Residual Network

Flow network G , flow f



Residual network G_f
(zero-capacity edges omitted)



The **amber** edge (v_2, v_3) with $c_f = 4$ lies on an augmenting path $p = s \rightarrow v_1 \rightarrow v_2 \rightarrow v_3 \rightarrow t$ (in blue in CLRS Fig. 24.4) — the bottleneck of p is exactly this edge.

Definition — Augmentation $f \oplus f'$

If f is a flow in G and f' is a flow in the residual network G_f , define $f \oplus f' : V \times V \rightarrow \mathbb{R}$ by

$$(f \oplus f')(u, v) = \begin{cases} f(u, v) + f'(u, v) - f'(v, u) & \text{if } (u, v) \in E, \\ 0 & \text{otherwise.} \end{cases}$$

Intuition. Flow on (u, v) increases by $f'(u, v)$ but decreases by $f'(v, u)$, because pushing flow on the reverse residual edge means *cancelling* flow in the original network.

Example (cancellation). 5 items go $u \rightarrow v$, 2 go $v \rightarrow u$. Net effect: equivalent to sending 3 item $u \rightarrow v$ and none back. This cancellation mechanism is essential to every max-flow algorithm.

Lemma 24.1: Augmentation Produces a Larger Flow

Lemma 24.1

Let $G = (V, E)$ be a flow network with flow f , let G_f be its residual network, and let f' be a flow in G_f . Then $f \oplus f'$ is a flow in G with value

$$|f \oplus f'| = |f| + |f'|.$$

This is the theoretical engine behind every augmenting-path algorithm: as long as we can find a flow f' with $|f'| > 0$ in the residual network, we can strictly increase the value of f while staying feasible in G .

Proof outline (next slide): (1) capacity constraint for $f \oplus f'$ on every edge of E ; (2) flow conservation at every internal vertex, which also yields the value identity.

Proof of Lemma 24.1 — Capacity Constraint

Fix $(u, v) \in E$. Since $(u, v) \in E$, we have $c_f(v, u) = f(u, v)$; since f' is a flow in G_f , $f'(v, u) \leq c_f(v, u) = f(u, v)$. Hence:

$$(f \oplus f')(u, v) = f(u, v) + f'(u, v) - f'(v, u) \geq f(u, v) + f'(u, v) - f(u, v) = f'(u, v) \geq 0.$$

For the upper bound, using nonnegativity of f' and $f'(u, v) \leq c_f(u, v) = c(u, v) - f(u, v)$:

$$(f \oplus f')(u, v) \leq f(u, v) + f'(u, v) \leq f(u, v) + c(u, v) - f(u, v) = c(u, v).$$

So $0 \leq (f \oplus f')(u, v) \leq c(u, v)$ — the capacity constraint holds. ■

Proof of Lemma 24.1 — Conservation & Value

For any $u \in V$, partition neighbors into $V_\ell(u) = \{v : (u, v) \in E\}$ and $V_e(u) = \{v : (v, u) \in E\}$ (disjoint, since G has no antiparallel edges). A term-by-term regrouping (CLRS eq. 24.5–24.6) shows:

$$\sum_v (f \oplus f')(u, v) - \sum_v (f \oplus f')(v, u) = \left(\sum_v f(u, v) - \sum_v f(v, u) \right) + \left(\sum_v f'(u, v) - \sum_v f'(v, u) \right)$$

- Setting $u = s$: the two parenthesized terms are exactly $|f|$ and $|f'|$ by definition, giving $|f \oplus f'| = |f| + |f'|$.
- Setting $u \notin \{s, t\}$: flow conservation for *both* f and f' makes *both* parenthesized terms 0, so $f \oplus f'$ also satisfies conservation at u .

Key Idea

Lemma 24.1 is exactly why residual-network reasoning works: legality (capacity + conservation) and additivity of value both survive augmentation.

Definition — Augmenting path

Given flow network G and flow f , an **augmenting path** p is a simple path from s to t in the residual network G_f .

The flow on every edge of p can increase by up to the smallest residual capacity along p :

$$c_f(p) = \min\{c_f(u, v) : (u, v) \text{ is on } p\}.$$

This quantity is the **residual capacity of the path** p .

Lemma 24.2

Define $f_p(u, v) = c_f(p)$ if (u, v) is on p , else 0. Then f_p is a flow in G_f with value $|f_p| = c_f(p) > 0$.

Proof idea: f_p trivially satisfies the capacity constraint by definition of $c_f(p)$ as the path minimum, and conservation holds because every internal vertex of the simple path has exactly one unit of f_p -flow entering and leaving (Exercise 24.2-7).

Corollary 24.3

Let f be a flow in G , p an augmenting path in G_f , and f_p as in Lemma 24.2. Then $f \oplus f_p$ is a flow in G with

$$|f \oplus f_p| = |f| + |f_p| > |f|.$$

Proof: immediate from Lemma 24.1 (additivity) and Lemma 24.2 ($|f_p| > 0$).

Worked example. Continuing the earlier network: the augmenting path $p = s \rightarrow v_1 \rightarrow v_2 \rightarrow v_3 \rightarrow t$ has residual capacity $c_f(p) = c_f(v_2, v_3) = 4$ (the bottleneck edge). Augmenting f by f_p increases the flow value from 19 to 23.

Corollary 24.3 & Example of Augmenting II

Ford–Fulkerson method

Start from $f \equiv 0$. Repeat: find *any* augmenting path p in G_f ; augment f by f_p . Stop when no augmenting path exists. We will soon prove this terminal flow is *maximum*.

Part II — Cuts & the Max-Flow Min-Cut Theorem

Certifying optimality without trying every flow

Definition — Cut

A **cut** (S, T) of $G = (V, E)$ is a partition of V into S and $T = V - S$ such that $s \in S$ and $t \in T$.

Net flow across the cut:

$$f(S, T) = \sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{u \in S} \sum_{v \in T} f(v, u).$$

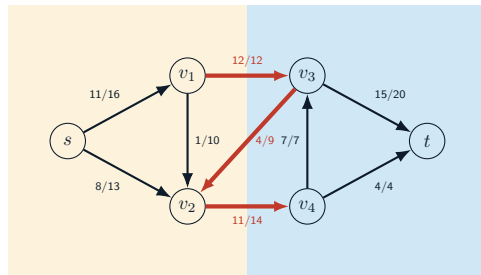
Capacity of the cut:

$$c(S, T) = \sum_{u \in S} \sum_{v \in T} c(u, v).$$

A crucial asymmetry

Net flow counts edges crossing **in both directions**; capacity counts **only** edges directed from the S -side to the T -side. This asymmetry is exactly what makes cuts useful as upper bounds on flow value.

Worked Example of a Cut



Cut (S, T) with $S = \{s, v_1, v_2\}$ (amber) and $T = \{v_3, v_4, t\}$ (blue). Edges crossing $S \rightarrow T$ are highlighted in red.

Net flow across the cut:

$$f(v_1, v_3) + f(v_2, v_4) - f(v_3, v_2) = 12 + 11 - 4 = 19.$$

Capacity of the cut ($S \rightarrow T$ edges only):

$$c(v_1, v_3) + c(v_2, v_4) = 12 + 14 = 26.$$

Note $f(S, T) = 19 = |f|$ — *exactly* the flow value! This is not a coincidence (Lemma 24.4).

Lemma 24.4: Every Cut Sees the Same Net Flow

Lemma 24.4

Let f be a flow in G with source s , sink t , and let (S, T) be *any* cut of G . Then the net flow across (S, T) equals the flow value:

$$f(S, T) = |f|.$$

Proof sketch. Start from the flow-conservation identity $\sum_v f(u, v) - \sum_v f(v, u) = 0$ for every $u \in S - \{s\}$, and add it to the definition of $|f| = \sum_v f(s, v) - \sum_v f(v, s)$, summed over all $u \in S - \{s\}$. After regrouping (splitting sums over $V = S \cup T$ and cancelling the terms internal to S), everything collapses to

$$|f| = \sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{u \in S} \sum_{v \in T} f(v, u) = f(S, T).$$



Corollary 24.5: Cuts Upper-Bound the Flow

Corollary 24.5

The value of any flow f in G is bounded from above by the capacity of *any* cut of G : $|f| \leq c(S, T)$ for every cut (S, T) .

Proof. By Lemma 24.4 and the capacity constraint ($f(v, u) \geq 0$):

$$|f| = f(S, T) = \sum_{u \in S} \sum_{v \in T} f(u, v) - \underbrace{\sum_{u \in S} \sum_{v \in T} f(v, u)}_{\geq 0} \leq \sum_{u \in S} \sum_{v \in T} f(u, v) \leq \sum_{u \in S} \sum_{v \in T} c(u, v) = c(S, T).$$

Immediate consequence

The value of a **maximum** flow is bounded above by the capacity of a **minimum** cut. This gives us a certificate strategy: if we ever exhibit a flow f and a cut (S, T) with $|f| = c(S, T)$, then f must be maximum *and* (S, T) must be minimum — no further search needed.

Theorem 24.6 — Max-Flow Min-Cut Theorem

If f is a flow in $G = (V, E)$ with source s and sink t , the following are equivalent:

1. f is a maximum flow in G .
2. The residual network G_f contains no augmenting path.
3. $|f| = c(S, T)$ for some cut (S, T) of G .

This is the theoretical anchor of the whole chapter:

- $(1) \Leftrightarrow (2)$ tells us exactly *when to stop* an augmenting-path algorithm.
- $(2) \Leftrightarrow (3)$ tells us *how to extract a certificate* (the min cut) once we stop.

We prove $(1) \Rightarrow (2) \Rightarrow (3) \Rightarrow (1)$, a cycle of implications, which establishes all three are equivalent.

Proof of Theorem 24.6: (1) $\Rightarrow$ (2)

Claim: if f is maximum, G_f has no augmenting path.

Proof (contrapositive via contradiction). Suppose f is maximum but G_f *does* have an augmenting path p . By Corollary 24.3, the flow $f \oplus f_p$ is a flow in G with

$$|f \oplus f_p| = |f| + |f_p| > |f|,$$

since $|f_p| = c_f(p) > 0$ by Lemma 24.2. This contradicts the assumption that f has maximum value. ■

Key Idea

This direction is the *easy* half and is exactly the correctness argument behind every augmenting-path algorithm: as long as an augmenting path exists, we are provably not yet optimal.

Proof of Theorem 24.6: (2) $\Rightarrow$ (3)

Suppose G_f has no path from s to t . Define

$$S = \{v \in V : \text{there is a path from } s \text{ to } v \text{ in } G_f\}, \quad T = V - S.$$

$s \in S$ trivially; $t \notin S$ since there's no s - t path in G_f — so (S, T) is a valid cut.

Take any $u \in S$, $v \in T$:

- If $(u, v) \in E$: we must have $f(u, v) = c(u, v)$ — otherwise $c_f(u, v) > 0$, putting $(u, v) \in E_f$ and forcing $v \in S$, a contradiction.
- If $(v, u) \in E$: we must have $f(v, u) = 0$ — otherwise $c_f(u, v) = f(v, u) > 0$, again forcing $v \in S$.
- If neither edge exists: $f(u, v) = f(v, u) = 0$ trivially.

So every $S \rightarrow T$ edge is *saturated* and every $T \rightarrow S$ edge carries *zero* flow. Hence

$$f(S, T) = \sum_{u \in S} \sum_{v \in T} c(u, v) - 0 = c(S, T),$$

and by Lemma 24.4, $|f| = f(S, T) = c(S, T)$. ■

Proof of Theorem 24.6: (3) $\Rightarrow$ (1), and Wrap-Up

By Corollary 24.5, $|f| \leq c(S, T)$ for every cut. If additionally $|f| = c(S, T)$ for some particular cut, then f cannot be improved upon: any flow f'' satisfies $|f''| \leq c(S, T) = |f|$, so f is maximum. ■

Algorithmic payoff

Combining all three: an augmenting-path algorithm that runs until G_f has no s - t path automatically:

- produces a **maximum** flow (by $2 \Rightarrow 1$), and
- exposes the reachable-set construction $S = \{v : s \rightsquigarrow v \text{ in } G_f\}$ as an explicit **minimum cut** witnessing optimality (by $2 \Rightarrow 3$).

This is precisely the correctness proof for Ford–Fulkerson, coming up next.

Part III — The Ford–Fulkerson Method

Turning the theorem into an algorithm

The Ford–Fulkerson Method

Why a “method” and not an “algorithm”? Because it leaves the choice of augmenting path unspecified — different choices give different concrete algorithms with different running times.

Procedure Ford-Fulkerson-Method(G, s, t)

- 1: initialize flow f to 0
- 2: **while** there exists an augmenting path p in the residual network G_f **do**
- 3: augment flow f along p
- 4: **end while**
- 5: **return** f

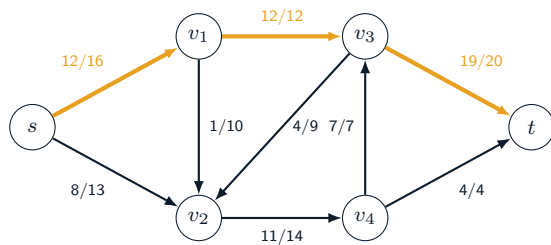
By Theorem 24.6, when the loop terminates (G_f has no augmenting path), f is guaranteed to be a **maximum** flow — regardless of which augmenting paths were chosen along the way.

The Concrete Ford-Fulkerson Procedure

Procedure Ford-Fulkerson(G, s, t)

```
1: for each edge  $(u, v) \in G.E$  do
2:    $(u, v).f = 0$ 
3: end for
4: while there exists a path  $p$  from  $s$  to  $t$  in the residual network  $G_f$  do
5:    $c_f(p) = \min\{c_f(u, v) : (u, v) \text{ is in } p\}$ 
6:   for each edge  $(u, v)$  in  $p$  do
7:     if  $(u, v) \in G.E$  then
8:        $(u, v).f = (u, v).f + c_f(p)$ 
9:     else
10:       $(v, u).f = (v, u).f - c_f(p)$  ▷  $(u, v)$  is a reverse residual edge
11:     end if
12:   end for
13: end while
14: return  $f$ 
```

Sample Run: Five Augmentations to Optimality I



Sample Run: Five Augmentations to Optimality II

The textbook's full trace (Fig. 24.6) starts from $f \equiv 0$ and finds five successive augmenting paths, reaching final value $|f^*| = 23$.

- Each iteration finds *some* path in G_f , computes its bottleneck $c_f(p)$, and pushes that much flow along it.
- One of the five augmenting paths uses a *reverse* residual edge — i.e., it **cancels** previously sent flow on (v_3, v_2) .
- After the fifth augmentation, G_f has no s - t path: by Theorem 24.6, f with $|f| = 23$ is optimal, and the corresponding reachable set gives an explicit minimum cut of capacity 23.

Running-Time Analysis (Naive Path Search)

Data structure. Maintain $G' = (V, E')$ with $E' = \{(u, v) : (u, v) \in E \text{ or } (v, u) \in E\}$; store both capacity and current flow per edge of G . Given flow f , the residual network's edges are exactly those $(u, v) \in E'$ with $c_f(u, v) > 0$.

- Finding an augmenting path by BFS/DFS in G_f : $O(V + E') = O(E)$ time.
- Each iteration of the while loop: $O(E)$ time (path-finding + $O(V)$ -length augmentation).
- Initialization (lines 1–2): $O(E)$.

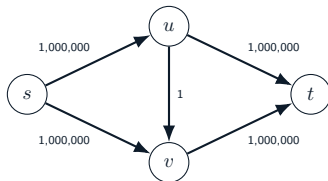
Running time — integer capacities

If all capacities are integers and f^* is a maximum flow, the flow value increases by *at least* 1 per iteration, so the while loop runs at most $|f^*|$ times. Total running time: $O(E |f^*|)$.

Caveat. With *irrational* capacities, an adversarial choice of augmenting paths can make the algorithm run forever, with the flow value converging to something *strictly less* than the maximum!

The Trouble With Naive Path Choice I

$O(E|f^*|)$ is *pseudo-polynomial*: it depends on the numeric value of f^* , not just on $|V|, |E|$. This can be catastrophic.



The true maximum flow has value 2,000,000 (paths $s \rightarrow u \rightarrow t$ and $s \rightarrow v \rightarrow t$). But if the algorithm alternately picks $s \rightarrow u \rightarrow v \rightarrow t$ and $s \rightarrow v \rightarrow u \rightarrow t$, each augmentation is limited by the bottleneck edge (u, v) with capacity 1:

- Flow increases by exactly **1 unit per iteration**.
- Total iterations: 2,000,000 — matching $O(E|f^*|)$ exactly, and *utterly impractical*.

The Trouble With Naive Path Choice II

Key Idea

The fix is not a smarter data structure — it's a smarter *rule* for choosing which augmenting path to use. That rule is Edmonds–Karp.

Part III (cont.) — The Edmonds–Karp Algorithm

Choosing augmenting paths wisely: BFS

The Edmonds–Karp Rule

Edmonds–Karp

Always choose the augmenting path p to be a **shortest** path from s to t in G_f (fewest edges, found by breadth-first search).

This single choice is enough to guarantee *polynomial* running time, independent of the numeric capacities.

Notation. Let $\delta_f(u, v)$ denote the shortest-path distance from u to v in G_f , where every edge has unit length. The whole analysis tracks how $\delta_f(s, v)$ evolves as f changes over the course of the algorithm.

Roadmap of the analysis:

- 1 Distances from s never decrease over time (Lemma 24.7).
- 2 Each edge can be “critical” on an augmenting path only $O(V)$ times (Theorem 24.8).
- 3 $\Rightarrow$ total number of augmentations is $O(VE)$, each costing $O(E)$: total $O(VE^2)$.

Lemma 24.7: Distances Increase Monotonically

Lemma 24.7

Running Edmonds–Karp on $G = (V, E)$: for every $v \in V - \{s, t\}$, the shortest-path distance $\delta_f(s, v)$ in G_f increases monotonically with each flow augmentation.

Proof (by contradiction). Suppose some augmentation (flow $f \rightarrow f'$) *decreases* some distance. Let v be a vertex with the *minimum* $\delta_{f'}(s, v)$ among those whose distance decreased, so $\delta_{f'}(s, v) < \delta_f(s, v)$. Let $p = s \rightsquigarrow u \rightarrow v$ be a shortest path to v in $G_{f'}$, so $(u, v) \in E_{f'}$ and

$$\delta_{f'}(s, u) = \delta_{f'}(s, v) - 1. \quad (*)$$

By minimality of v 's decrease, u 's distance did *not* decrease:

$$\delta_{f'}(s, u) \geq \delta_f(s, u). \quad (**)$$

Proof of Lemma 24.7 (continued)

Case A: $(u, v) \in E_f$ **already.** Then by the triangle inequality in G_f , and $(*)$, $(**)$:

$$\delta_f(s, v) \leq \delta_f(s, u) + 1 \stackrel{(**)}{\leq} \delta_{f'}(s, u) + 1 \stackrel{(*)}{=} \delta_{f'}(s, v),$$

contradicting $\delta_{f'}(s, v) < \delta_f(s, v)$.

Case B: $(u, v) \notin E_f$ **but** $(u, v) \in E_{f'}$. This can only happen if the augmentation increased flow from v to u , i.e., (v, u) was on the (shortest) augmenting path in G_f . Since prefixes of shortest paths are shortest paths, this means $\delta_f(s, v) = \delta_f(s, u) - 1$. Then:

$$\delta_f(s, v) = \delta_f(s, u) - 1 \stackrel{(**)}{\leq} \delta_{f'}(s, u) - 1 \stackrel{(*)}{=} \delta_{f'}(s, v) - 2,$$

so $\delta_{f'}(s, v) \geq \delta_f(s, v) + 2 > \delta_f(s, v)$ — again a contradiction.

Both cases contradict our assumption, so no such v exists. ■

Definition — Critical edge

An edge (u, v) on an augmenting path p is **critical** if its residual capacity equals the path's bottleneck: $c_f(p) = c_f(u, v)$.

- After augmenting along p , every critical edge on p **disappears** from the residual network (its residual capacity drops to 0).
- Every augmenting path has **at least one** critical edge (the bottleneck itself).
- An edge (u, v) can only become critical *again* after its residual capacity becomes positive again — which requires flow on (v, u) to *decrease*, i.e., (v, u) itself appears on some later augmenting path.

Key Idea

Bounding *how many times* each of the $O(E)$ edges can become critical directly bounds the total number of augmentations, since each augmentation retires at least one critical edge.

Theorem 24.8: $O(VE)$ Total Augmentations I

Theorem 24.8

Running Edmonds–Karp on $G = (V, E)$, the total number of augmentations performed is $O(VE)$.

Proof. Fix an edge (u, v) that can appear in some residual network. The *first* time it is critical, since p is a shortest path,

$$\delta_f(s, v) = \delta_f(s, u) + 1.$$

The edge then disappears until (v, u) appears on a later augmenting path, at flow f' ; at that point (again, shortest path):

$$\delta_{f'}(s, u) = \delta_{f'}(s, v) + 1.$$

Theorem 24.8: $O(VE)$ Total Augmentations II

Using Lemma 24.7 ($\delta_f(s, v) \leq \delta_{f'}(s, v)$):

$$\delta_{f'}(s, u) = \delta_{f'}(s, v) + 1 \geq \delta_f(s, v) + 1 = \delta_f(s, u) + 2.$$

So between consecutive times (u, v) is critical, $\delta_f(s, u)$ increases by **at least 2**.

- $\delta_f(s, u) \geq 0$ always, and since $u \neq t$ (it lies on an augmenting path, which ends at t), any path from s to u has at most $|V| - 2$ edges.
- So $\delta_f(s, u)$ ranges over at most $|V|/2$ “slots” of size 2, meaning (u, v) can be critical at most

$$1 + \frac{(|V| - 2)}{2} = \frac{|V|}{2}$$

times over the whole execution.

Theorem 24.8: $O(VE)$ Total Augmentations III

There are $O(E)$ pairs of vertices that can have a residual edge between them, so the total number of critical-edge events is $O(VE)$. Since every augmentation retires ≥ 1 critical edge, the total number of augmentations is $O(VE)$. ■

Total running time of Edmonds–Karp

Each augmentation costs $O(E)$ (BFS to find the shortest path, then update flow along it). Total:

$$O(VE) \cdot O(E) = O(VE^2).$$

This bound is **polynomial** in $|V|, |E|$ alone — no dependence on capacity values.

Ford–Fulkerson vs. Edmonds–Karp: Summary I

	Ford–Fulkerson (naive)	Edmonds–Karp
Path selection	Any augmenting path	Shortest augmenting path (BFS)
Per-iteration cost	$O(E)$	$O(E)$
# iterations	up to $ f^* $ (pseudo-polynomial)	$O(VE)$ (polynomial)
Total time	$O(E f^*)$	$O(VE^2)$
Termination guaranteed?	Only for rational/integer capacities	Always (finite, polynomial)

Take-away for algorithm engineering

The *method* (augment until no augmenting path) is correct by Theorem 24.6 no matter how you pick paths. But the *choice rule* determines whether the running time is merely finite or genuinely polynomial. This is a recurring theme: greedy correctness $\neq$ greedy efficiency.

Part IV — Maximum Bipartite Matching

Reducing a combinatorial problem to max flow

The Maximum Bipartite Matching Problem

Definitions

Given undirected $G = (V, E)$, a **matching** is $M \subseteq E$ such that no two edges of M share an endpoint. A vertex touched by an edge of M is **matched**; a **maximum matching** has the largest possible $|M|$.

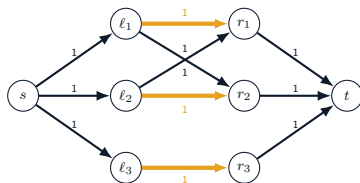
We restrict to **bipartite** graphs: $V = L \cup R$ (disjoint), every edge goes between L and R , and every vertex has at least one incident edge.

Practical applications (resource allocation):

- Assigning a set L of machines to a set R of tasks that must run simultaneously — edge (u, v) means machine u can perform task v ; a maximum matching keeps as many machines busy as possible.
- Assigning students to projects, jobs to applicants, servers to requests, frequencies to transmitters — any “compatibility” bipartite structure with a one-to-one constraint.

On the surface this looks nothing like a flow problem — but it reduces to one *exactly*.

From Bipartite Graph to Flow Network



Construction of $G' = (V', E')$:

- New source s , sink t ; $V' = V \cup \{s, t\}$.
- Direct every edge of E from L to R .
- Add (s, u) for every $u \in L$, and (v, t) for every $v \in R$.
- **Every** edge of E' gets **unit capacity**.

Since every vertex has an incident edge, $|E| \geq |V|/2$, so $|E'| = |E| + |V| \leq 3|E|$, i.e., $|E'| = \Theta(E)$.

The amber edges above (a matching of size 3) correspond to unit flows along $s \rightarrow \ell_i \rightarrow r_i \rightarrow t$.

Lemma 24.9: Matchings $\leftrightarrow$ Integer-Valued Flows

Lemma 24.9

Let $G = (V, E)$, $V = L \cup R$, with corresponding flow network G' .

- If M is a matching in G , there is an integer-valued flow f in G' with $|f| = |M|$.
- Conversely, if f is an integer-valued flow in G' , then $M = \{(u, v) \in E : f(u, v) > 0\}$ is a matching in G with $|M| = |f|$.

Forward direction. Given matching M , define $f(s, u) = f(u, v) = f(v, t) = 1$ for every $(u, v) \in M$, and 0 elsewhere. Each edge of M corresponds to one unit of flow along the vertex-disjoint path $s \rightarrow u \rightarrow v \rightarrow t$; these paths only share s, t , so capacity and conservation hold trivially. The net flow across the cut $(L \cup \{s\}, R \cup \{t\})$ is exactly $|M|$, so by Lemma 24.4, $|f| = |M|$.

Proof of Lemma 24.9: the Converse

Given an integer-valued flow f in G' , let $M = \{(u, v) : u \in L, v \in R, f(u, v) > 0\}$.

Why M is a matching. Each $u \in L$ has exactly one entering edge, (s, u) , with capacity 1 — so at most 1 unit of flow enters u , and by conservation at most 1 unit leaves. Since f is **integer-valued**, that unit (if present) leaves along a *single* edge to some $v \in R$. So each $u \in L$ is the tail of at most one edge in M . A symmetric argument (using the single edge (v, t) with capacity 1) shows each $v \in R$ is the head of at most one edge in M . Hence M is a valid matching.

Why $|M| = |f|$. For (u, v) with $u \in L, v \in R$: $f(u, v) = 1$ if $(u, v) \in M$, else 0. So the net flow across the cut $(L \cup \{s\}, R \cup \{t\})$ equals $|M|$; by Lemma 24.4, $|f| = f(L \cup \{s\}, R \cup \{t\}) = |M|$. ■

From Lemma 24.9 to an Algorithm

Lemma 24.9 shows a **bijective correspondence in value** between matchings in G and integer-valued flows in G' . Two remaining facts (developed fully with the *integrality theorem* later in the flow theory, stated here for completeness) close the loop:

- Because every capacity in G' is an **integer** (indeed, 0 or 1), the Ford–Fulkerson method — which only ever adds or subtracts residual capacities, themselves always integers — produces an integer-valued maximum flow.
- Hence a **maximum flow** in G' corresponds via Lemma 24.9 to a **maximum matching** in G , and its value is exactly $|M^*|$.

Running time

Run Edmonds–Karp (or even naive Ford–Fulkerson) on G' . Since capacities are 0/1, the maximum flow value is at most $\min(|L|, |R|) \leq |V|$, so naive Ford–Fulkerson already runs in $O(E|f^*|) = O(VE)$ time using $|E'| = \Theta(E)$. This matches the bound promised at the start of the section.

Maximum Bipartite Matching: Big Picture

Why this reduction matters

This is the paradigm case of **reducing a combinatorial problem to max flow**: we did not invent a new algorithm for matching — we recognized that the *integrality* of unit-capacity flow networks makes flows and matchings interchangeable, then reused all of the machinery already built: residual networks, augmenting paths, and the max-flow min-cut theorem.

Consequences for resource allocation, more generally:

- Any bipartite “compatibility + one-to-one assignment” problem (machines/tasks, workers/shifts, ads/slots) is solvable in $O(VE)$ time by this reduction.
- By Theorem 24.6, the min cut in G' is a certificate — e.g., a set of tasks provably unmatchable given the compatibility structure (a König/Hall-type deficiency argument).
- A specialized, faster algorithm (Hopcroft–Karp, achieving $O(E\sqrt{V})$) exists for this specific problem — exploiting the bipartite unit-capacity structure further. That’s a preview of the next lecture’s material.

Summary

Tying it all together for exam preparation

The chain of ideas end to end

- 1 **Flow network + flow**: capacities, conservation, value $|f|$.
- 2 **Residual network** G_f : lets us represent “undoing” flow via reverse edges; $f \oplus f'$ combines flows additively (Lemma 24.1).
- 3 **Augmenting path**: an s - t path in G_f ; using it strictly increases flow value (Corollary 24.3).
- 4 **Cut** (S, T) : any cut’s capacity upper-bounds every flow’s value (Corollary 24.5).
- 5 **Max-Flow Min-Cut Theorem**: no augmenting path $\Leftrightarrow$ maximum flow $\Leftrightarrow$ flow value equals *some* cut’s capacity.
- 6 **Ford–Fulkerson**: repeatedly augment; correct by the theorem; $O(E|f^*|)$ time, possibly exponential in input size.
- 7 **Edmonds–Karp**: augment along BFS shortest paths; $O(VE)$ augmentations (via monotonic distances + critical edges), $O(VE^2)$ total — *polynomial*.
- 8 **Bipartite matching**: unit-capacity reduction to max flow; integrality $\Rightarrow$ matching value = max-flow value; $O(VE)$ time.